

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Sydney pursuant to Part VB of the Copyright Act 1968 (**the Act**). The material in this communication may be subject to copyright under the Act. Any further copying or communication of this material by you may be the subject of copyright protection under the Act.

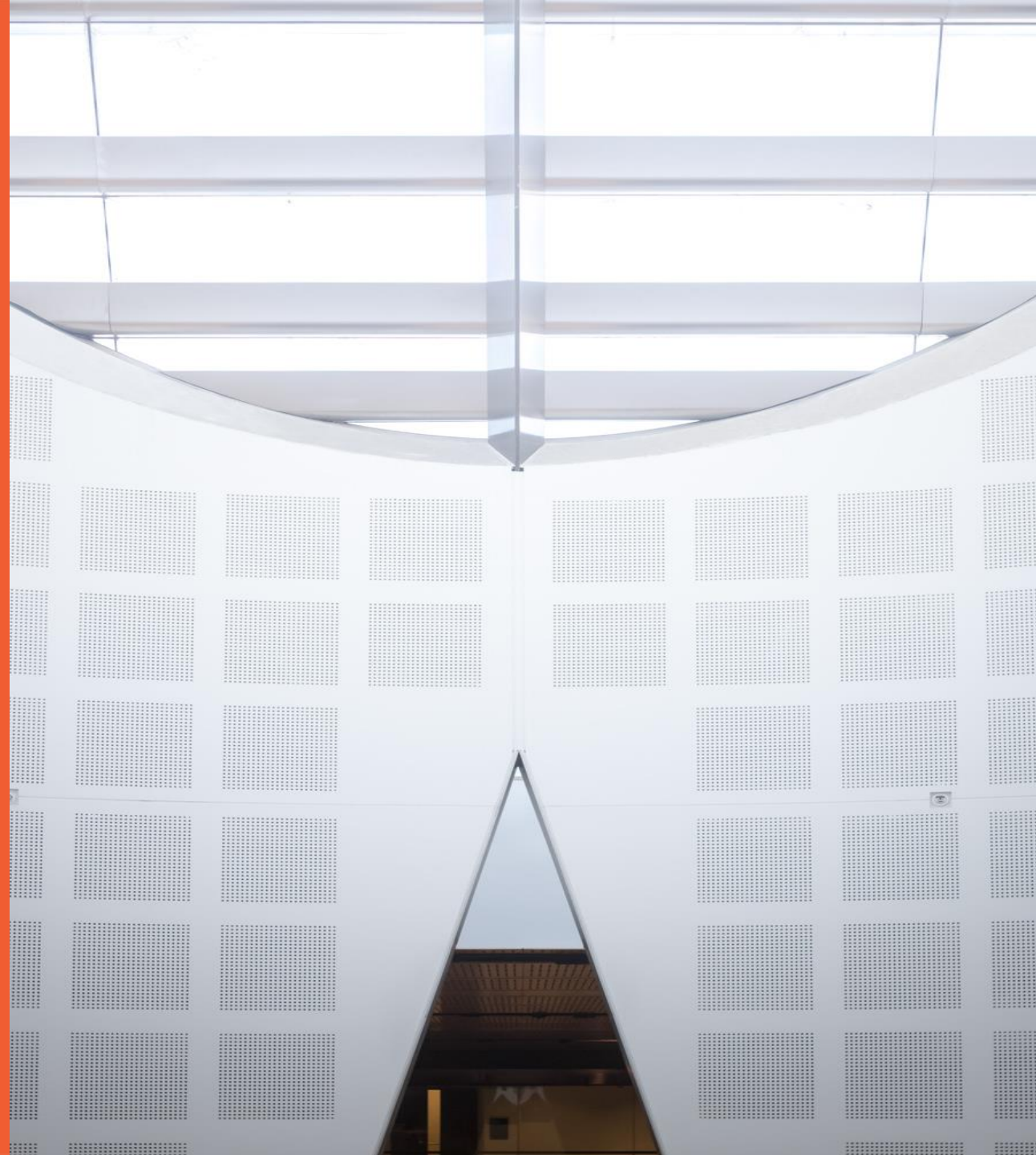
Do not remove this notice.

COMPx270: Randomised and
Advanced Algorithms
Lecture 6: Hashing and Friends

Clément Canonne
School of Computer Science



THE UNIVERSITY OF
SYDNEY

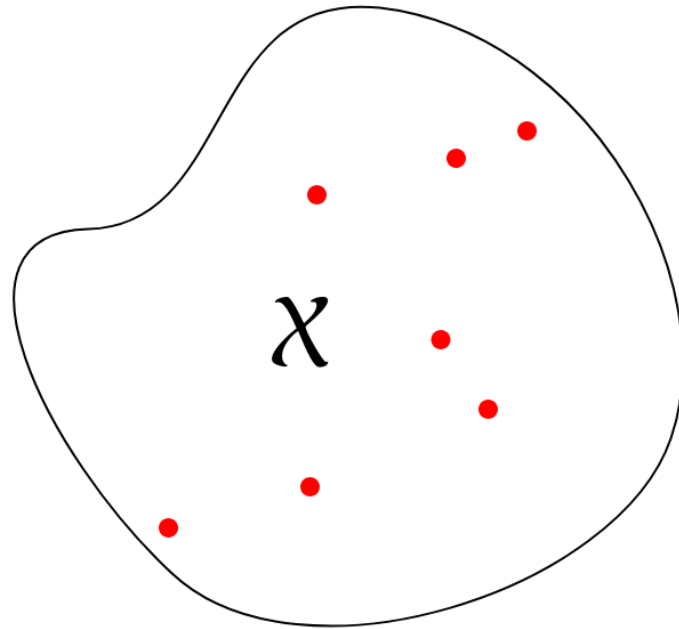


A question ?

A SID is of the form 450687816 (9 digits, each between 0 and 9). How many distinct SIDs are there?

How many distinct students have there even been at U Syd?

Dictionaries (Maps, Associative arrays...)

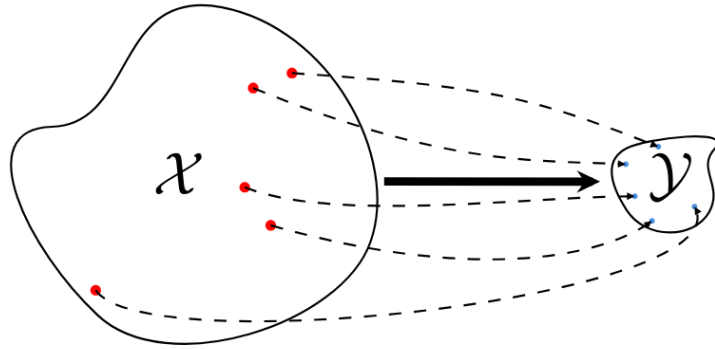


	List	Array	BST
Insert			
Lookup			
Remove			
Space			

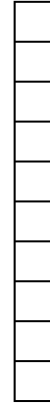
Important detour: data representation

IOIO
IOIO

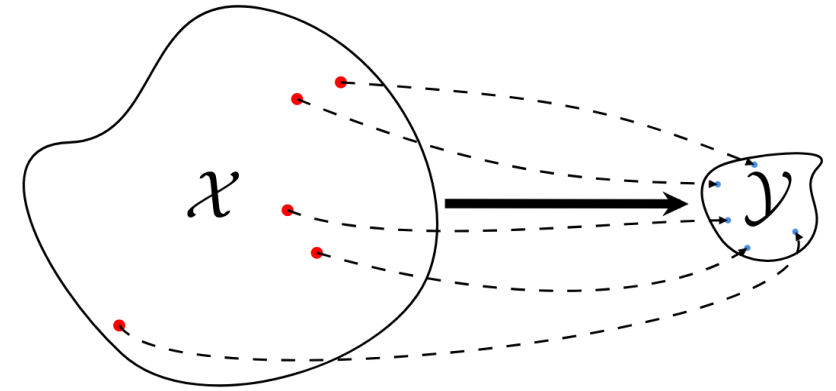
Hash tables



A



Hash tables: what is random?



Hash tables: hash families

Definition 31.1. A family of functions $\mathcal{H} \subseteq \{h: \mathcal{X} \rightarrow \mathcal{Y}\}$ is a *universal hash family*, if, for every $x, x' \in \mathcal{X}$ with $x \neq x'$,

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(x')] \leq \frac{1}{|\mathcal{Y}|}$$

where the probability is over the uniformly random choice of $h \in \mathcal{H}$.

Hash tables: hash families

Definition 31.1. A family of functions $\mathcal{H} \subseteq \{h: \mathcal{X} \rightarrow \mathcal{Y}\}$ is a *universal hash family*, if, for every $x, x' \in \mathcal{X}$ with $x \neq x'$,

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(x')] \leq \frac{1}{|\mathcal{Y}|}$$

where the probability is over the uniformly random choice of $h \in \mathcal{H}$.

Lemma 31.1. *Every strongly universal hash family is also a universal hash family. Moreover, there exist universal hash families which are not strongly universal.*

Hash tables: hash families

Definition 31.1. A family of functions $\mathcal{H} \subseteq \{h: \mathcal{X} \rightarrow \mathcal{Y}\}$ is a *universal hash family*, if, for every $x, x' \in \mathcal{X}$ with $x \neq x'$,

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(x')] \leq \frac{1}{|\mathcal{Y}|}$$

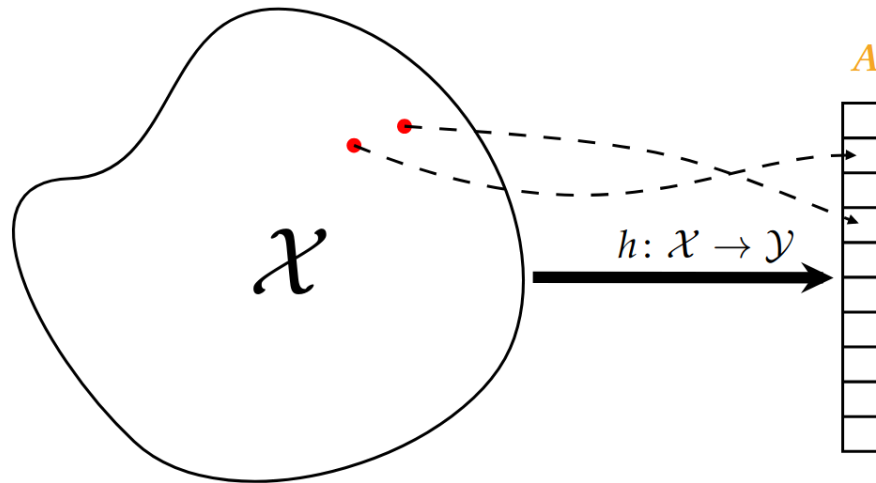
where the probability is over the uniformly random choice of $h \in \mathcal{H}$.

Theorem 32. Fix a prime number p . For given integers a, b , define the function $h_{a,b}: \mathbb{Z}_p \rightarrow [m']$ by

$$h_a(x) = (ax + b \bmod p) \bmod m', \quad x \in \mathbb{Z}_p$$

and let $\mathcal{H} = \{h_{a,b}\}_{1 \leq a < p, 0 \leq b < p}$. Then $\mathcal{H}$ is a universal hash family of size $|\mathcal{H}| \leq 2^{2 \log_2 p}$.

Hash tables: the data structure



 Function h
chosen once and for
all **when creating the
data structure**

Hash tables: no collisions? 🎂

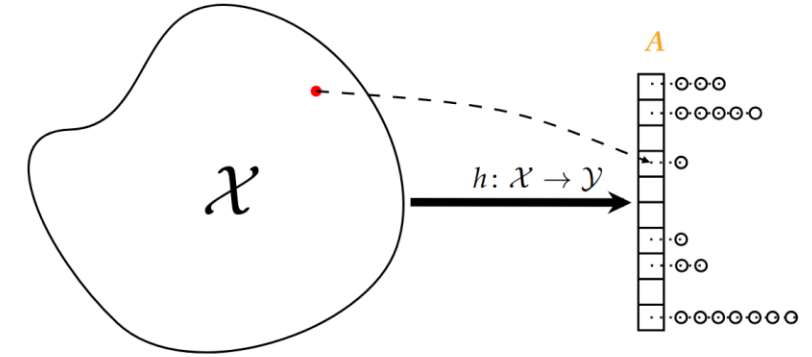
Hash tables: ~~no~~ collisions 🌟

Handling collisions: separate chaining

INSERT(x): call $A[h(x)].\text{INSERT}(x)$

LOOKUP(x): return $A[h(x)].\text{LOOKUP}(x)$

REMOVE(x): call $A[h(x)].\text{REMOVE}(x)$



Handling collisions: open addressing

INSERT(x):

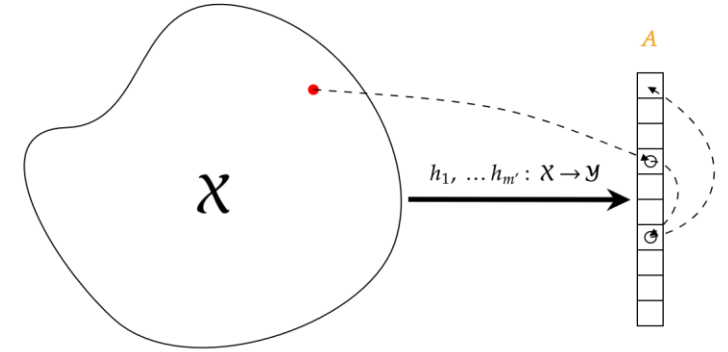
```
for all  $1 \leq t \leq m'$  do
  if  $A[h_t(x)] = x$  then
    return ▷ Already there
  else if  $A[h_t(x)] = \emptyset$  or  $A[h_t(x)] = \perp$  then
     $A[h_t(x)] \leftarrow x$  ▷ Insert it in the first available bucket
  return
```

LOOKUP(x):

```
for all  $1 \leq t \leq m'$  do
  if  $A[h_t(x)] = x$  then
    return yes
  else if  $A[h_t(x)] = \emptyset$  then
    return no ▷ If  $x$  was present, it'd have been found earlier
```

REMOVE(x):

```
for all  $1 \leq t \leq m'$  do
  if  $A[h_t(x)] = x$  then
     $A[h_t(x)] \leftarrow \perp$  ▷ Special symbol to indicate there was something before
  else if  $A[h_t(x)] = \emptyset$  then
    return ▷ If  $x$  was present, it'd have been found earlier
```



Handling collisions: open addressing

Theorem. Assuming **stuff** ($\dagger$), the expected time complexity of LOOKUP is

$$O\left(\frac{1}{1 - \alpha}\right).$$

Handling collisions: open addressing (linear probing)

Handling collisions: open addressing (linear probing)

Theorem (Knuth'62). Under some **reasonable** assumption, the expected time complexity of LOOKUP is

$$O\left(\frac{1}{(1 - \alpha)^2}\right) .$$

Handling collisions: open addressing (cuckoo hashing)



LOOKUP(x):

```
if  $A_1[h_1(x)] = x$  or  $A_2[h_2(x)] = x$  then
    return yes
else
    return no
```

REMOVE(x):

```
if  $A_1[h_1(x)] = x$  then
     $A_1[h_1(x)] \leftarrow \emptyset$ 
else if  $A_2[h_2(x)] = x$  then
     $A_2[h_2(x)] \leftarrow \emptyset$ 
```

Handling collisions: open addressing (cuckoo hashing)



INSERT(x):

if $A_1[h_1(x)] = \emptyset$ then

$A_1[h_1(x)] \leftarrow x$

else if $A_2[h_2(x)] = \emptyset$ then

$A_2[h_2(x)] \leftarrow x$

else

$T \leftarrow 0$

▷ The evictions start

$x' \leftarrow A_1[h_1(x)]$

$A_1[h_1(x)] \leftarrow x$

while $T < T_{\max}$ do

x' goes to $A_2[h_2(x')]$, and if there was something there, that element now needs to move, etc.

$T \leftarrow T + 1$

Handling collisions: open addressing (cuckoo hashing)



Theorem (Pagh–Flemming'01). Cuckoo hashing achieves worst-case time $O(1)$ for LOOKUP and REMOVE, and **expected** time $O(1)$ for INSERT.

Hash tables: summary

- The hash function h is **not** a "truly random function", but is chosen **once and for all**, at random  , from a **small** hash family $\mathcal{H}$
- h is then kept as part of the data structure: $O(\log |\mathcal{H}|)$ bits
- The **data** itself is **not** assumed to be random: only the choice of h from $\mathcal{H}$ is
- **No silver bullet**: things are only good in expectation (or with high probability)

Can we do better? Bloom filters!

Faster, less space, **but...** LOOKUP is sometimes **wrong**

Perspective: why does it work so well? 🙋

"Hashing is fundamental to many algorithms and data structures widely used in practice. For theoretical analysis of hashing, there have been two main approaches. First, one can **assume that the hash function is truly random**, mapping each data item independently and uniformly to the range. **This idealized model is unrealistic** because a truly random hash function requires an exponential number of bits to describe. Alternatively, one can provide rigorous bounds on performance when **explicit families of hash functions are used, such as 2-universal or $O(1)$ -wise independent families**. For such families, performance guarantees are often noticeably weaker than for ideal hashing.

In practice, however, it is commonly observed that simple hash functions, including 2-universal hash functions, perform as predicted by the idealized analysis for truly random hash functions."

📄 *"Why Simple Hash Functions Work: Exploiting the Entropy in a Data Stream."*
Mitzenmacher and Vadhan, 2008